WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

仓颉版S3客户端介绍

主讲人：王文斌

01 仓颉的使用及感受

02 S3客户端设计 & 仓颉特性使用

03 S3客户端使用



关于仓颉

填补国产编程语言的空白

https://developer.huawei.com/consumer/cn/cangjie



仓颉 Cangjie

关于普元：仓颉生态合作伙伴

低代码平台

低代码开发平台

低代码开发平台社区版 NEW

微服务平台

开发运维一体化平台

数字化应用解决方案

数智化应用研发平台

企业级客户敏感数据保护系统

基于数字基座的生产系统方案

智能化的数据中台

元数据管理平台

主数据管理平台

数据资产管理软件

数据资源目录软件

数据共享服务平台

数字化应用解决方案

集团企业数字化运营监管平台

一站式服务大厅

校本数据中心平台

高校数据治理方案

信创中间件

应用服务器中间件

企业服务总线中间件

消息中间件

文件传输中间件

工作流中间件

数据交换平台中间件

数字化应用解决方案

企业数字化流程应用中心

3

系列

20+

产品

仓颉 Cangjie

仓颉使用感受



- 扩展 (extend)
- 元编程 (宏)
- 仓颉语言的性能分析工具(cjprof)
- 严格的包管理机制

熟悉之后

使用初期



- 实现和继承用了<:
- 类型转换
- let, var 的使用
- 标准库接口不熟
- 没有三目运算符
- IDE不够智能等

仓颉 Cangjie

仓颉使用感受： @Builder的替代实现

```
public class PutObjectRequest <: AbstractS3Request {
    public PutObjectRequest(
        requestConfig!: ?S3RequestConfig = None,
        public let bucket!: String,
        public let key!: String,
        public let acl!: ?String = None,
        public let cacheControl!: ?String = None,
        public let contentDisposition!: ?String = None,
        public let contentEncoding!: ?String = None,
        public let contentLanguage!: ?String = None,
        public let contentLength!: ?Int64 = None,
        public let contentMD5!: ?String = None,
        public let contentType!: ?String = None,
        public let checksumAlgorithm!: ?String = None,
        public let checksumCRC32!: ?String = None,
        public let checksumCRC32C!: ?String = None,
        public let checksumSHA1!: ?String = None,
```

let配上主构造函数以及命名参数，可以代替@Builder方式创建复杂对象

```
let putReq = PutObjectRequest(
    bucket: bucket1,
    key: key,
    expires: DateTime.now(),
    metadata: HashMap([("k1", "v1"), ("k2", "v2")])
)
let putObjectRsp = s3.putObject(putReq, S3Content.fromFile("./object.txt"))
println("=== PutObject(key=${key})")
```

```
PutObjectRequest req = PutObjectRequest.builder() //
        .bucket("junit") //
        .key("key_06") //
        .expires(Instant.now()) //
        .metadata(ImmutableMap.of("k1", "v1", "k2", "v2")) //
        .build();
PutObjectResponse rsp = s3.putObject(req, new File("./b.txt").toPath());
System.out.println(rsp);
```

仓颉 Cangjie

仓颉使用感受

- 仓颉语言博采众长，属于重业务开发的静态类型语言
- 保持一个初学者的心态去了解、学习和使用仓颉语言
- 多看文档，多看其他库源码，多看Issue

仓颉 Cangjie

仓颉S3客户端

s3client4cj



介绍

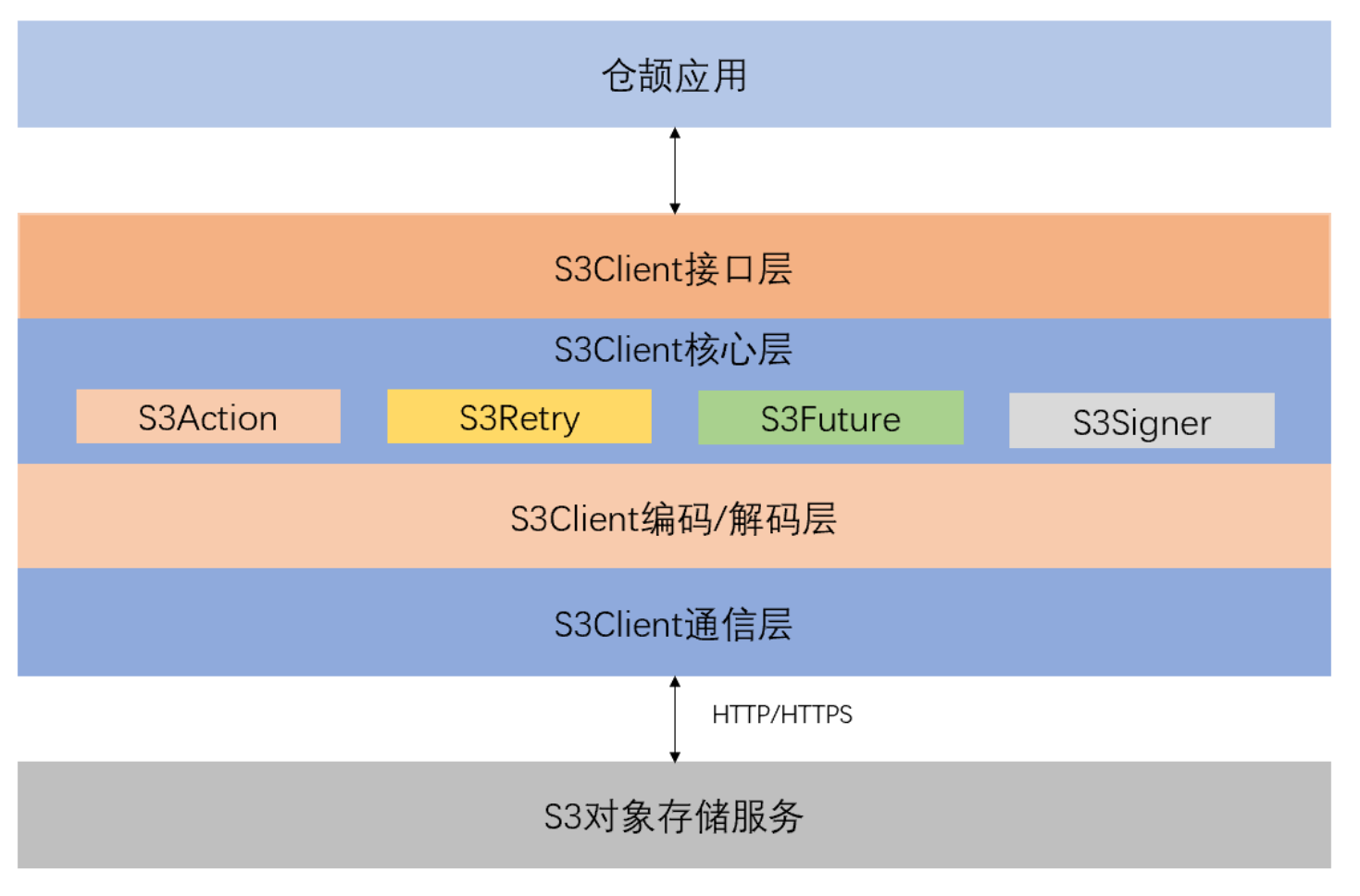
AWS S3 的仓颉客户端实现, 参考Java实现 `software.amazon.awssdk:s3:2.21.21`

特性

- 🚀 支持 AWS Signature Version 2
- 🚀 支持 AWS Signature Version 4
- 🚀 支持 S3 Service 的错误响应报文
- 🚀 支持 Bucket 相关操作接口
- 🚀 支持 Object 相关操作接口
- 🚀 支持同步/异步编程接口
- 🚀 支持分段上传
- 🚀 支持自动分页
- 🚀 支持预签名 URL (Presigned URL)
- 🚀 支持 MD5/SHA1/SHA256 的 Checksum
- 🚀 支持 SHA1/SHA256 的 Trailing Checksum
- 🚀 支持 Metric
- 🚀 支持重试
- 🚀 支持 Waiter

仓颉 Cangjie

仓颉S3客户端：总体架构

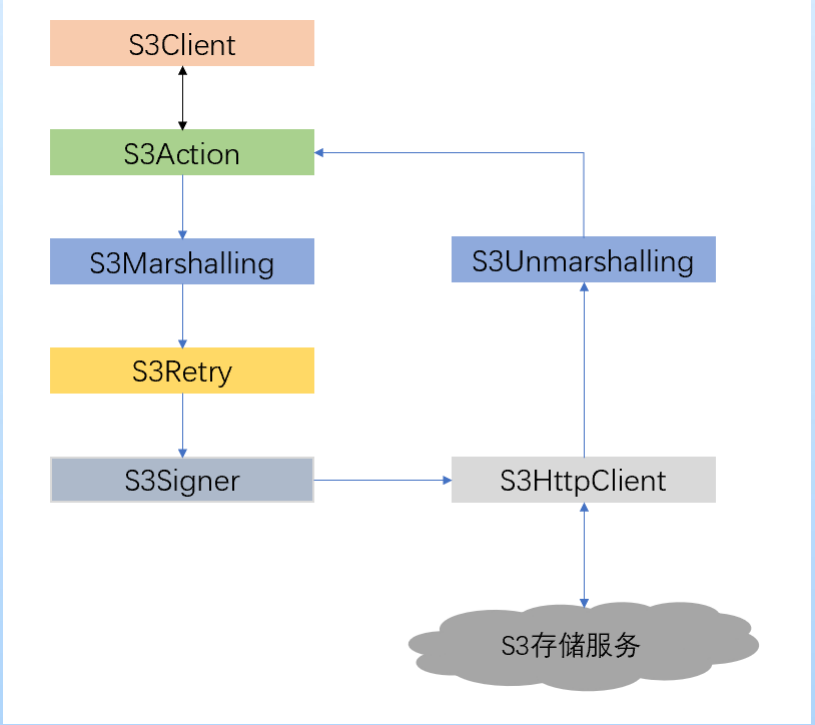


仓颉 Cangjie

仓颉S3客户端：Action处理流程

支持的 S3 Action 列表

- HeadBucket (p. 203)
- CreateBucket (p. 34)
- ListBuckets (p. 233)
- DeleteBucket (p. 50)
- DeleteBucketAnalyticsConfiguration (p. 52)
- DeleteBucketCors (p. 54)
- DeleteBucketEncryption (p. 56)
- DeleteBucketIntelligentTieringConfiguration (p. 58)
- DeleteBucketInventoryConfiguration (p. 60)
- DeleteBucketLifecycle (p. 62)
- DeleteBucketMetricsConfiguration (p. 64)
- DeleteBucketOwnershipControls (p. 66)
- DeleteBucketPolicy (p. 68)
- DeleteBucketReplication (p. 70)
- DeleteBucketTagging (p. 72)
- DeleteBucketWebsite (p. 74)
- PutBucketAccelerateConfiguration (p. 280)
- PutBucketAcl (p. 283)
- PutBucketCors (p. 294)



仓颉 Cangjie

仓颉特性：扩展 (extend)

```
extend String {
    public func toBool(): Bool {
        return Bool.parse(this.toAsciiLower())
    }

    public func toInt64(): Int64 {
        return Int64.parse(this)
    }

    public func toInt32(): Int32 {
        return Int32.parse(this)
    }

    public func toDateRfc822(): DateTime {
        return DateUtils.toDateRfc822(this)
    }

    public func toDateIso8601(): DateTime {
        return DateUtils.toDateIso8601(this)
    }
}
```

```
headers.getFirst(S3Constants.LAST_MODIFIED)?.toDateRfc822(),
 headers.getFirst(S3Constants.CONTENT_LENGTH)?.toInt64(),
```

仓颉 Cangjie

仓颉特性：元编程 (macro)

```
@MJsonEnumType["index|create"]
public class OpType {}

@MJsonEnumType["true|false|wait_for"]
public class Refresh {}

@MJsonEnumType["internal|external|external_gte|force"]
public class VersionType {}

@MJsonEnumType["created|updated|deleted|not_found|noop"]
public class Result {}
```

```
Generate MJsonEnumType: OpType,
public class OpType <: ToString & JsonDeserializable < OpType > {

    public static let Index = OpType("index")

    public static let Create = OpType("create")


    private static let _ALL = {
        =>
        let all = HashMap < String, OpType >()
        all.put(Index.value, Index)
        all.put(Create.value, Create)

        all
    }()

    public static func of(value: String): OpType {
        return _ALL.get(value).getOrThrow({ => throw IllegalArgumentException("Unrecognized value: ${value}") })
    }

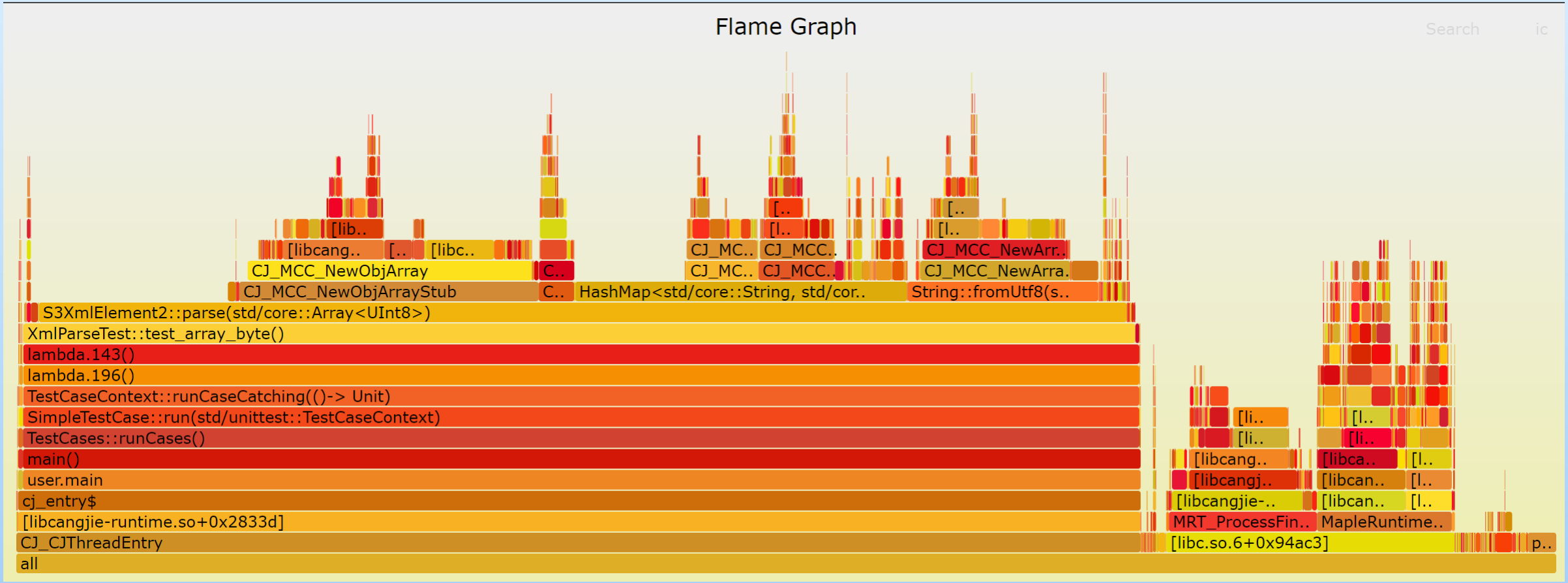
    public let value: String

    private OpType(value: String) {
        this.value = value
    }

    public func toString(): String {
        return "OpType(${value})"
    }

    public static func fromJson(r: JsonReader): OpType {
        if(let Some(token) <- r.peek()) {
            return match(token) {
                case JsonString => OpType.of(r.readValue < String >())
                case _ => throw IllegalArgumentException(
                "Expected JsonToken is [JsonString], but the actual is [${ToStringUtils.toString(token)}]")
            }
        } else {
            throw NoneValueException("JsonReader peek a none token")
        }
    }
}
```

仓颉特性：性能分析工具 (cjprof)



仓颉S3客户端使用：创建客户端

```
from s3client4cj import s3client.*

main() {
    let s3 = S3Client.builder()
            .credentials("${accessKeyId}", "${secretAccessKey}")
            .trailingChecksum(true)
            .build()
    ....
    s3.close()
}
```

支持的自定义属性：

- 支持自定义 HttpClient
- 支持自定义 Retryer
- 支持Signature切换（V2/V4）
- ...

指定访问端点	endpoint()	String
HTTP Client 实现	httpClient()	s3client.core.S3HttpClient
签名方式	signer()	s3client.core.S3Signer
是否强制路径方式	forcePathStyle()	Bool
重试	retryer()	s3client.core.S3Retryer<s3client.core.S3HttpResponse>
是否记录错误的 HTTP Response	loggingErrorResponse()	Bool
是否使用尾随校验和	trailingChecksum()	Bool

仓颉 Cangjie

仓颉S3客户端使用：Demo

https://gitcode.com/Cangjie-TPC/s3-sdk/tree/main/demo

```
main(): Int64 {
    let s3 = S3Client.builder() //
        .endpoint("http://127.0.0.1:9000") //
        .forcePathStyle(true) // 本地 minio 需要
        .credentials("7TqWZaaRE8IZuzjXKctd", "Oe16N435sZZtXM9qXs59Yo6v5JvOAmNvGZTvpzkX") //
        .build()

    let bucket = "async-oldsix"

    if (isBucketExists(s3, bucket)) {
        println("===> Bucket '${bucket}' is exists")
        s3.close()
        return 0
    }

    let createReq = CreateBucketRequest(bucket: bucket)
    s3.createBucketAsync(createReq).thenAsync<Unit>() {
        rsp, ex => match (rsp) {
            case Some(createRsp) => println(createRsp)
            case None => println("===> EX: ${ex.getOrThrow()}")
        }
    }
    println("After CreateBucketAsync")
    sleep(Duration.second * 1)
    println("After sleep 1 second")
    s3.close()
    return 0
}
```

仓颉 Cangjie

未来规划

- 稳定代码，Bug修改，预计年底左右发布1.0.0版本；后续准备根据用户使用情况及其反馈的Issue发版

- 欢迎大家给我们提交 PR，欢迎给我们提交 Issue，欢迎参与任何形式的贡献。

  https://gitcode.com/Cangjie-TPC/s3-sdk/overview

仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!

